



ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

UNIVERSITY OF SCIENCE AND TECHNOLOGY - UD

HỆ THỐNG VI XỬ LÝ NÂNG CAO Giao thức USART

TƯ DUY - SÁNG TẠO - NHÂN ÁI

NỘI DUNG

Giới thiệu về USART

Các thanh ghi USART

Cách tính tốc độ Baud Rate

Cấu hình cho USART

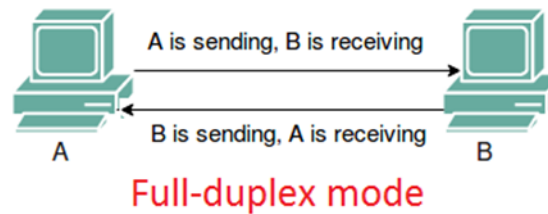
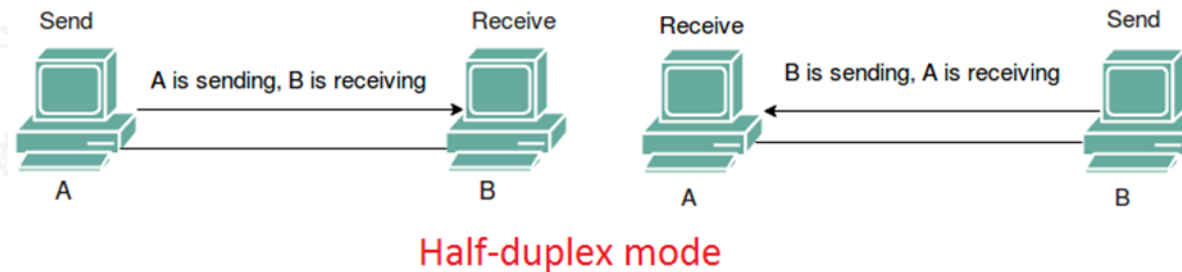
Ngắt USART

Các ví dụ

GIỚI THIỆU GIAO THỨC USART

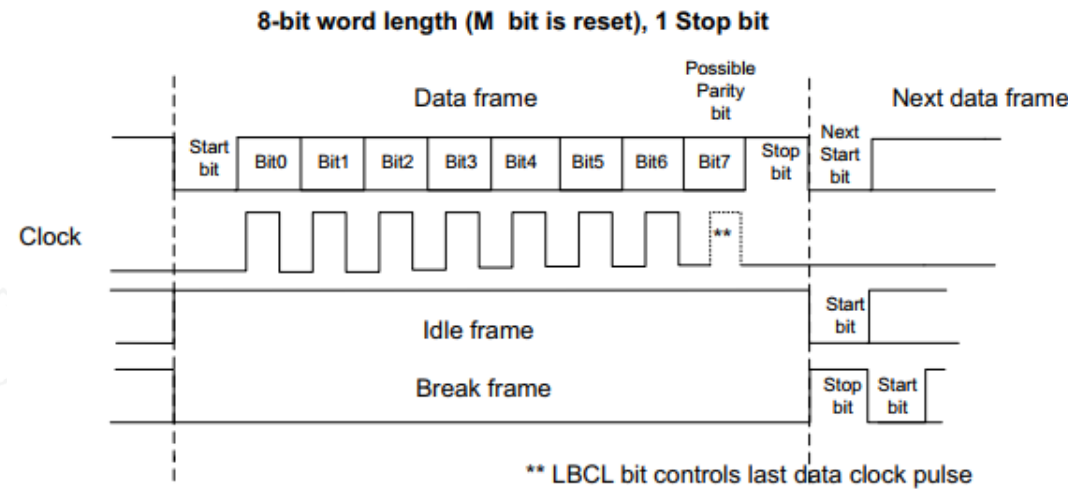
USART – Universal Synchronous/Asynchronous Receiver/ Transmitter là một chuẩn truyền thông nối tiếp (half-duplex) được tích hợp trong rất nhiều thiết bị, module, vi điều khiển, vi xử lý, cảm biến, các hệ thống tự động hóa... Chuẩn truyền thông USART sử dụng 2 mức logic theo chuẩn TTL tức thể hiện logic 0 (0V) và logic 1 (3-5V).

Dữ liệu truyền đi có thể là đồng bộ hoặc không đồng bộ. Ở vi điều khiển chúng ta chỉ sử dụng cách truyền là không đồng bộ tức dữ liệu sẽ được đóng gói thành các khung truyền và có tốc độ truyền (BAUD - bps) được chuẩn hóa và phải giống nhau tốc độ Baud.



GIỚI THIỆU GIAO THỨC USART

Khung truyền của giao thức USART

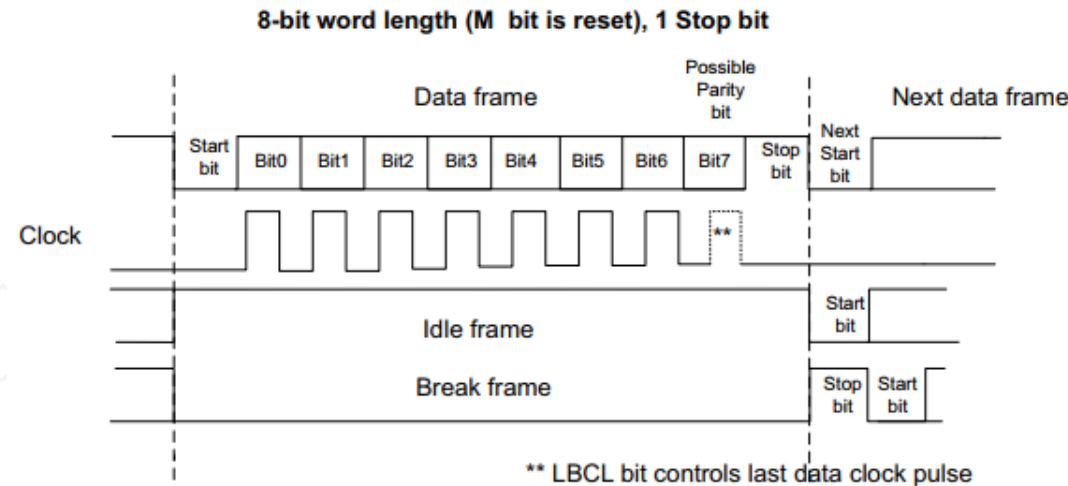


Một khung truyền dữ liệu của bộ USART gồm 1 Start bit, 1 data word (8 hoặc 9 bit tùy người dùng cấu hình), 0,5/1/1,5/2 stops bits. Độ dài của 1 data word được lập trình bằng cách ghi mức logic 1 hoặc 0 vào M bit của thanh ghi USART_CR1.

- **Baud rate (tốc độ baud):** số bit truyền nhận trong 1s, cả 2 thiết bị truyền nhận cho nhau phải cùng 1 tốc độ baud.

GIỚI THIỆU GIAO THỨC USART

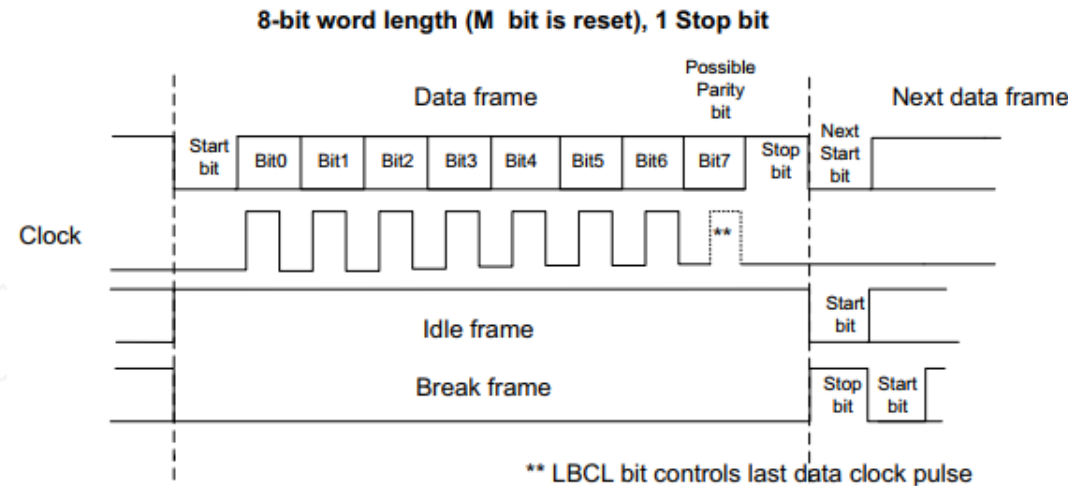
Khung truyền của giao thức USART



- **Start bit:** Start-bit còn được gọi là bit đồng bộ hóa được đặt trước dữ liệu thực tế. Để bắt đầu truyền dữ liệu, truyền UART kéo đường dữ liệu từ mức điện áp cao (1) xuống mức điện áp thấp (0), thông báo cho thiết bị nhận có một gói dữ liệu sắp được truyền đến.
- **Data:** Các bit dữ liệu bao gồm dữ liệu thực được truyền từ người gửi đến người nhận. Độ dài khung dữ liệu có thể nằm trong khoảng 5 & 8. Nếu bit chẵn lẻ được sử dụng thì chiều dài khung dữ liệu có thể dài 9 bit.

GIỚI THIỆU GIAO THỨC USART

Khung truyền của giao thức USART



- **Parity bit:** Bit chẵn lẻ cho phép người nhận đảm bảo liệu dữ liệu được nhận có đúng hay không. Nếu tổng số bit 1 không khớp với chế độ parity đã chọn(even hoặc odd) thì dữ liệu đã bị lỗi trong quá trình truyền.
- **Stop bit:** Bit dừng được đặt ở phần cuối của gói dữ liệu. Bit này được dùng để báo cho các thiết bị nhận rằng các bit đã được truyền xong.

Khung truyền của giao thức USART

Chế độ

Asynchronous (UART)

Synchronous

STM32F103C8 có **3 mô-đun USART**:

- ✓ USART1 (nằm ở bus APB2)
- ✓ USART2, USART3 (nằm ở bus APB1)

Đặc điểm

Giao tiếp không có xung clock, chuẩn UART (TX/RX).

Có xung clock (thêm chân CK), cho phép truyền nhanh hơn.

Các chân USART thông dụng

USART	TX	RX	CK (Clock)	Ghi chú
USART1	PA9	PA10	PA8	Tốc độ cao (APB2 – 72 MHz)
USART2	PA2	PA3	PA4	Giao tiếp phổ biến, dùng với USB-TTL
USART3	PB10	PB11	PB12	Dùng thêm cổng nối tiếp thứ 2

CÁC THANH GHI USART



Thanh ghi cho phép cấp xung ngoại vi RCC_APB2ENR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART1EN	Res.	SPI1EN	TIM1EN	ADC2EN	ADC1EN	Reserved			IOPEEN	IOPDEN	IOPCEN	IOPBEN	IOPAEN	Res.
	rw		rw	rw	rw	rw				rw	rw	rw	rw	rw	rw

- ❖ Bit USART1EN (RCC_APB2ENR[14]) =1: Cho phép xung clock cho USART1
- ❖ Bit AFIOEN (RCC_APB2ENR[0]) =1: Cho phép xung clock cho AFIO.

CÁC THANH GHI USART



Thanh ghi cho phép cấp xung ngoại vi RCC_APB1ENR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		DAC EN	PWR EN	BKP EN	CAN2 EN	CAN1 EN	Reserved		I2C2 EN	I2C1 EN	UART5E N	UART4E N	USART3 EN	USART2 EN	Res.
		rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWD GEN	Reserved					TIM7 EN	TIM6 EN	TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN
rw	rw			rw						rw	rw	rw	rw	rw	rw

- ❖ Bit USART2EN (RCC_APB2ENR[17]) =1: Cho phép xung clock cho USART2.
- ❖ Bit USART3EN (RCC_APB2ENR[18]) =1: Cho phép xung clock cho USART3.
- ❖ Bit USART4EN (RCC_APB2ENR[19]) =1: Cho phép xung clock cho USART4.
- ❖ Bit USART5EN (RCC_APB2ENR[20]) =1: Cho phép xung clock cho USART5.

CÁC THANH GHI USART

Thanh ghi điều khiển thứ 1 USART_CR1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		UE	M	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	RWU	SBK
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- ❖ Bit RE (USART_CR1[2]) =1: Cho phép nhận dữ liệu.
- ❖ Bit TE (USART_CR1[3]) =1: Cho phép truyền dữ liệu.
- ❖ Bit RXNEIE (USART_CR1[5]) =1: Cho phép ngắt USART sau khi nhận từng kí tự.
- ❖ Bit TXEIE (USART_CR1[7])=1: Cho phép ngắt USART sau khi truyền từng kí tự.
- ❖ Bit M (USART_CR1[12]) : Khai báo độ dài dữ liệu
- ❖ Bit UE (USART_CR1[13]) =1: Cho phép USART hoạt động.

CÁC THANH GHI USART

Thanh ghi trạng thái USART_SR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						CTS	LBD	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
						rc_w0	rc_w0	r	rc_w0	rc_w0	r	r	r	r	r

- ❖ Bit RXNE (USART_SR[5]) =1: Dữ liệu sẵn sàng được đọc.
- ❖ Bit TC (USART_SR[6]) =1: Quá trình truyền hoàn thành.
- ❖ Bit TXE (USART_SR[7]) =1: Dữ liệu đã được truyền đi và thanh ghi đã sẵn sàng để nhận dữ liệu mới.

CÁC THANH GHI USART

Thanh ghi dữ liệu USART_DR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							DR[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw

- ❖ Thanh ghi USART_DR có chức năng lưu trữ dữ liệu truyền đi hoặc nhận được.

CÁC THANH GHI USART



Thanh ghi Baud rate USART_BRR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIV_Mantissa[11:0]												DIV_Fraction[3:0]			
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

- ❖ Thanh ghi USART_BRR có chức năng cấu hình tốc độ truyền dữ liệu của giao tiếp USART.
- ❖ DIV_Mantissa[11:0]: 12 bits này xác định phần nguyên của hệ số chia (USARTDIV) dùng để tính toán tốc độ Baud.
- ❖ DIV_Fraction[3:0]: 4 bits này xác định phần thập phân của hệ số chia (USARTDIV) dùng để tính toán tốc độ Baud.

CÁCH TÍNH TỐC ĐỘ BAUD RATE

Cách tính tốc độ Baud rate

$$\text{USART_DIV} = \frac{f_{ck}}{16 * \text{Baud rate}}$$

Trong đó:

- ❖ USART_DIV: Hệ số chia
- ❖ f_{ck} : Tần số xung clock cấp cho USART
- ❖ Baud Rate: Tốc độ truyền dữ liệu

Ví dụ: Tốc độ truyền dữ liệu của USART2 là 9600

USART2 được cấp xung clock bởi RCC_APB1ENR có $f_{ck} = 36\text{MHz}$

$$\text{USART_DIV} = \frac{36 * 10^6}{16 * 9600} = 234.375$$

DIV_Fraction = $16 * 0.375 = 6 \rightarrow$ Chuyển sang hệ 16 là 0x06

DIV_Mantissa = 234 $\rightarrow$ Chuyển sang hệ 16 là 0xEA

Vậy giá trị trong thanh ghi USART_BRR là 0xEA6

CẤU HÌNH USART

Các bước cấu hình giao thức USART

- **Bước 1:** Cấp xung clock cho AFIOEN.
- **Bước 2:** Cho phép XUNG USART hoạt động.
- **Bước 3:** Cấu hình các chân của USART.
- **Bước 4:** Cấu hình tốc độ truyền dữ liệu Baud Rate.
- **Bước 5:** Định nghĩa độ dài dữ liệu truyền nhận.
- **Bước 6:** Cấu hình số lượng bit stop.
- **Bước 7:** Cho phép truyền dữ liệu.
- **Bước 8:** Cho phép nhận dữ liệu.
- **Bước 9:** Cho phép USART hoạt động.

```
//STEP UART
```

```
//B1:Enable the Alternative Function for PINs
```

```
RCC->APB2ENR |= (1<<0); //Cap xung clock cho AFIOEN
```

```
//B2:Enable USART2
```

```
RCC->APB1ENR |= (1<<17); //Cap xung clock cho USART2
```

```
//B3:Enable the related Pins
```

```
init_GP(PA,2,OUT50,O_AF_PP); //PA2:TX-->output
```

```
init_GP(PA,3,IN,I_PP); //PA3:RX-->input
```

```
//B4:Setup the baud rate: 9600 bits/s
```

```
USART2->BRR = 0xEA6;
```

CẤU HÌNH USART

Các bước cấu hình giao thức USART

- **Bước 1:** Cấp xung clock cho AFIOEN.
- **Bước 2:** Cho phép XUNG USART hoạt động.
- **Bước 3:** Cấu hình các chân của USART.
- **Bước 4:** Cấu hình tốc độ truyền dữ liệu Baud Rate.
- **Bước 5:** Định nghĩa độ dài dữ liệu truyền nhận.
- **Bước 6:** Cấu hình số lượng bit stop.
- **Bước 7:** Cho phép truyền dữ liệu.
- **Bước 8:** Cho phép nhận dữ liệu.
- **Bước 9:** Cho phép USART hoạt động.

//B5:Define the word length

USART2->CR1 &=~(1<<12); //bit M=0:1 Start bit, 8 Data bits

//B6:Config the number of stop bits

USART2->CR2 &=~(1<<12); // 1 Stop bit

USART2->CR2 &=~(1<<13);

//B7:Enable USART

USART2->CR1 |= (1<<13); //bit UE=1

B8:Enable UART transmit

USART2->CR1 |= (1<<3); //bit TE=1

//B9:Enable UART receive

USART2->CR1 |= (1<<2); //bit RE=1

CÁC VÍ DỤ



Quá trình truyền dữ liệu

Transmitter có thể gửi các dữ liệu có độ dài 8 hoặc 9 bit, tùy thuộc vào trạng thái của bit M. Khi bit kích hoạt truyền (TE) được đặt, dữ liệu trong thanh ghi dịch chuyển được xuất ra trên chân TX và các xung đồng hồ tương ứng được xuất ra trên chân CK.

Một byte dữ liệu từ đường bus sẽ được lưu trong bộ đệm TDR. Khi bit cho phép truyền (TE) được set, dữ liệu sẽ được đưa xuống thanh ghi Transmit Shift Register và bit có trọng số thấp nhất (LSB) sẽ được gửi đi đầu tiên trên chân TX.

Sau khi truyền đi một byte, thanh ghi TDR sẽ được làm trống và byte tiếp theo có thể được truyền đi. Ở chế độ này dữ liệu của thanh ghi USART_DR chính là dữ liệu của bộ đệm TDR.

CÁC VÍ DỤ

Quá trình truyền 1 byte dữ liệu

❖ Các bước truyền một byte dữ liệu USART

- Bước 1: Đợi thanh ghi truyền dữ liệu trống bằng cách kiểm tra bit TXE trong thanh ghi USART_SR set lên 1.
- Bước 2: Đặt byte dữ liệu cần truyền vào thanh ghi USART_DR.
- Bước 3: Đợi việc truyền hoàn tất bằng bit TC trong thanh ghi USART_SR set lên 1.

❖ Hàm truyền một byte dữ liệu USART

```
void UART_TX(unsigned short uart,char c)
{
    if(uart ==1)
    {
        while((USART1 ->SR & (1<<7))==0x00) {}; //bit TXE
        USART1->DR=c;
        USART1->SR &=~(1<<7); //bit TXE
        while((USART1->SR & (1<<6))==0) {}; //bit TC
    }else if(uart==2){
        while((USART2 ->SR & (1<<7))==0) {}; //bit TXE
        USART2->DR=c;
        USART2->SR &=~(1<<7); //bit TXE
        while((USART2 ->SR & (1<<6))==0) {}; //bit TC
    }
}
```

CÁC VÍ DỤ

Quá trình nhận 1 byte dữ liệu

❖ Hàm nhận 1 byte dữ liệu

USART có thể nhận các từ dữ liệu có độ dài 8 hoặc 9 bit, tùy thuộc vào bit M trong thanh ghi USART_CR1.

Khi có dữ liệu được truyền từ thiết bị bên ngoài, bit có trọng số thấp nhất của frame data đầu tiên được truyền qua chân RX của MCU và lưu vào thanh ghi Receive Shift Register. Khi một byte đã nhận xong, bit RXNE (Read data register not empty) set lên 1, dữ liệu chứa trong thanh ghi Receive Shift Register được đưa đến bộ đệm RDR, dữ liệu lúc này đã sẵn sàng để đọc.

❖ Các bước nhận một byte dữ liệu USART

- Bước 1: Đợi thanh ghi nhận dữ liệu trống bằng cách chờ cho bit RXNE trong thanh ghi USART_SR set lên 1.
- Bước 2: Đọc dữ liệu nhận được từ thanh ghi USART_DR.

CÁC VÍ DỤ

Quá trình nhận 1 byte dữ liệu

❖ Hàm nhận 1 byte dữ liệu

```
char UART_RX(unsigned short uart)
{
    char c;
    if(uart == 1)
    {
        while((USART1 ->SR & (1<<5)) == 0x00) {}; //bit RXNE
        c=USART1->DR;
        USART1->SR &= ~(1<<5); //bit RXNE
    } else if(uart == 2){
        while((USART2 ->SR & (1<<5)) == 0x00) {}; //bit RXNE
        c=USART2->DR;
        USART2->SR &= ~(1<<5); //bit RXNE
    }
    return c;
}
```

CÁC VÍ DỤ

Quá trình truyền nhiều byte dữ liệu

❖ Các bước truyền nhiều byte dữ liệu USART

- Bước 1: Tạo một mảng chứa dữ liệu cần truyền đi.
- Bước 2: Đợi thanh ghi truyền dữ liệu trống bằng cách kiểm tra bit TXE trong thanh ghi USART_SR set lên 1.
- Bước 3: Đặt byte dữ liệu cần truyền vào thanh ghi USART_DR.
- Bước 4: Đợi việc truyền hoàn tất bằng bit TC trong thanh ghi USART_SR set lên 1.
- Bước 5: Tăng con trỏ lên 1 đơn vị và lặp lại cho tới khi truyền xong.

CÁC VÍ DỤ

Quá trình truyền nhiều byte dữ liệu

❖ Hàm truyền nhiều byte dữ liệu

```
void UART_SendData(unsigned short usart,uint8_t *pTxBuffer,uint32_t len)
{
    uint32_t i;
    if(usart==1)
    {
        for(i=0;i<len;i++)
        {
            while((USART1->SR &(1<<7))==0){} //bit TXE
            USART1->DR = (uint8_t)(*pTxBuffer & 0xFF);
            pTxBuffer++;
        }
        while((USART1->SR &(1<<6))==0){} //bit TC
    }else if(usart==2)
    {
        for(i=0;i<len;i++)
        {
            while((USART2->SR &(1<<7))==0){} //bit TXE
            USART2->DR = (uint8_t)(*pTxBuffer & 0xFF);
            pTxBuffer++;
        }
        while((USART2->SR &(1<<6))==0){} //bit TC
    }
}
```


CÁC VÍ DỤ

Quá trình nhận nhiều byte dữ liệu

❖ Các bước nhận nhiều byte dữ liệu USART

- Bước 1: Tạo mảng chứa dữ liệu nhận được.
- Bước 2: Đợi thanh ghi nhận dữ liệu trống bằng cách chờ cho bit RXNE trong thanh ghi USART_SR set lên 1.
- Bước 3: Đọc dữ liệu nhận được từ thanh ghi USART_DR
- Bước 4: Tăng con trỏ lên 1 đơn vị và lặp lại cho tới khi nhận xong.

CÁC VÍ DỤ

Quá trình nhận nhiều byte dữ liệu

❖ Hàm nhận nhiều byte dữ liệu

```
void UART_ReceiveData(unsigned short usart,uint8_t *pRxBuffer,uint32_t len)
{
    uint32_t i;
    if(usart==1)
    {
        for(i=0;i<len;i++)
        {
            while((USART1->SR &(1<<5))==0){} //bit RXNE
            *pRxBuffer = (uint8_t)(USART1->DR & (uint8_t)0xFF);
            pRxBuffer++;
        }
    }else if(usart ==2)
    {
        for(i=0;i<len;i++)
        {
            while((USART2->SR &(1<<5))==0){} //bit RXNE
            *pRxBuffer = (uint8_t)(USART2->DR & (uint8_t)0xFF);
            pRxBuffer++;
        }
    }
}
```

NGẮT USART



Các sự kiện ngắt và cờ ngắt của USART

Sự kiện ngắt	Cờ sự kiện ngắt	Bit cho phép ngắt
Thanh ghi dữ liệu truyền rỗng	TXE	TXEIE
Cờ CTS (Clear To Send)	CTS	CTSIE
Truyền hoàn thành	TC	TCIE
Dữ liệu nhận sẵn sàng để đọc	RXNE	RXNEIE
Phát hiện lỗi tràn dữ liệu	ORE	
Phát hiện đường truyền rỗi	IDLE	IDLEIE
Lỗi chẵn lẻ	PE	PEIE
Cờ Break (ngắt kết nối)	LBD	LBDIE
Cờ nhiễu, lỗi tràn dữ liệu và lỗi khung trong giao tiếp đa bộ đệm	NE hoặc ORE hoặc FE	EIE(1)

NGẮT USART

Các bước cấu hình ngắt USART khi nhận dữ liệu

- Bước 1: Cho phép ngắt khi nhận dữ liệu bằng cách set bit RXNEIE trong thanh ghi USART_CR1.
- Bước 2: Cho phép ngắt toàn cục.
- Bước 3: Viết hàm xử lý ngắt.

❖ Cấu hình ngắt USART

```
// Enable UART RXNE interrupt
USART2->CR1 |= (1 << 5); // bit RXNEIE:
__disable_irq();
NVIC_EnableIRQ(USART2_IRQn);
__enable_irq();
```

NGẮT USART

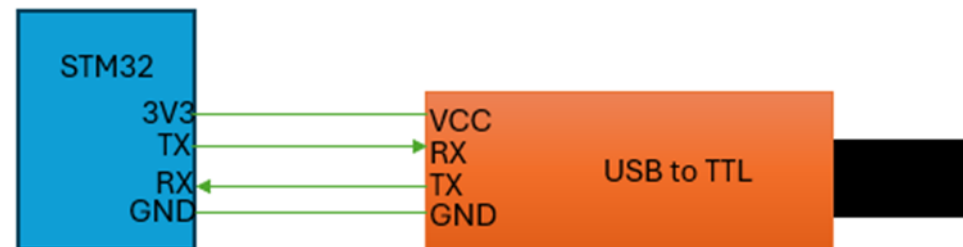
Các bước cấu hình ngắt USART khi nhận dữ liệu

❖ Hàm xử lý ngắt USART

```
void USART2_IRQHandler()
{
    uint8_t receivedByte = USART2->DR;
    if (count < sizeof(pRxBuffer)) {
        pRxBuffer[count] = receivedByte;
        count++;
        if (receivedByte == '@') {
            count = 0;
        }
    } else {
        count = 0;
    }
}
```


Giao tiếp giữa STM32C8T6 với laptop qua USART

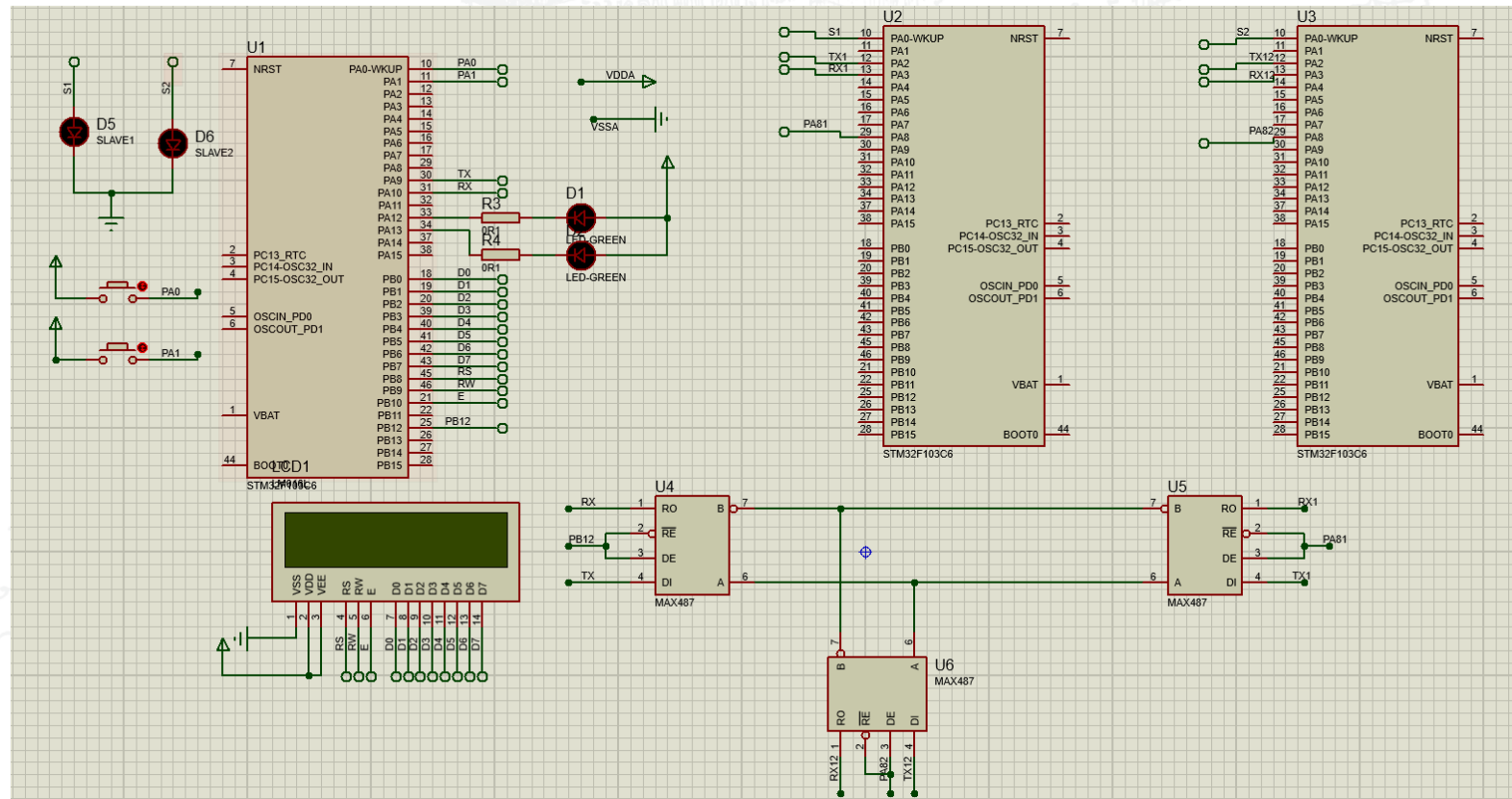
Vì Laptop của chúng ta sử dụng giao tiếp USB mà STM32 sử dụng giao thức UART nên chúng ta không thể kết nối trực tiếp từ STM lên máy tính để giao tiếp được. Chính vì thế ta cần 1 module có thể chuyển đổi từ USB sang USART hay còn gọi là **USB to TTL** hoặc **USB to COM** (sử dụng cho cổng RS232).



Sơ đồ kết nối USB to TTL và STM32

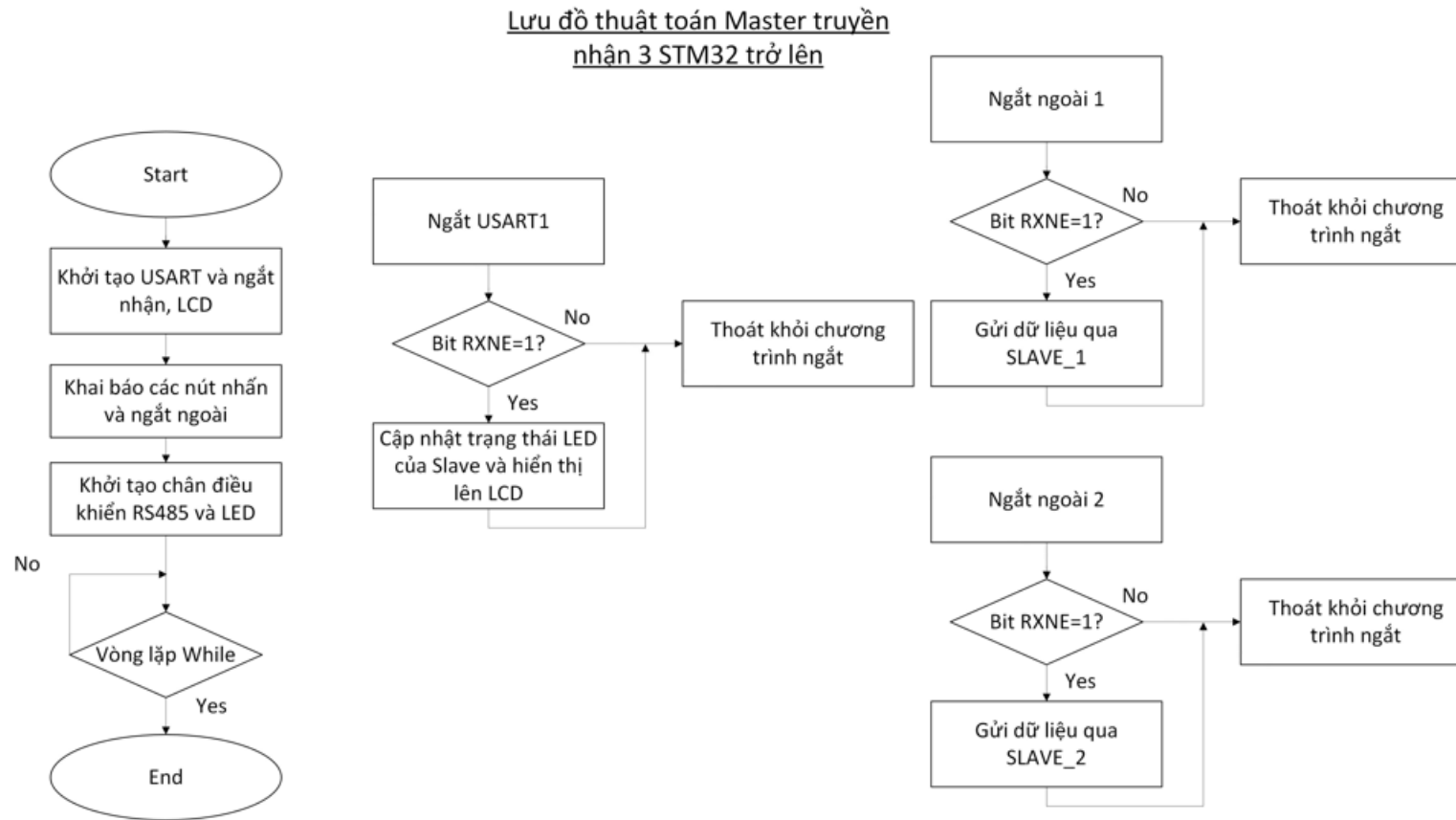
Giao tiếp giữa 1 Master và 2 Slave sử dụng RS485

❖ Sơ đồ mô phỏng



Giao tiếp giữa 1 Master và 2 Slave sử dụng RS485

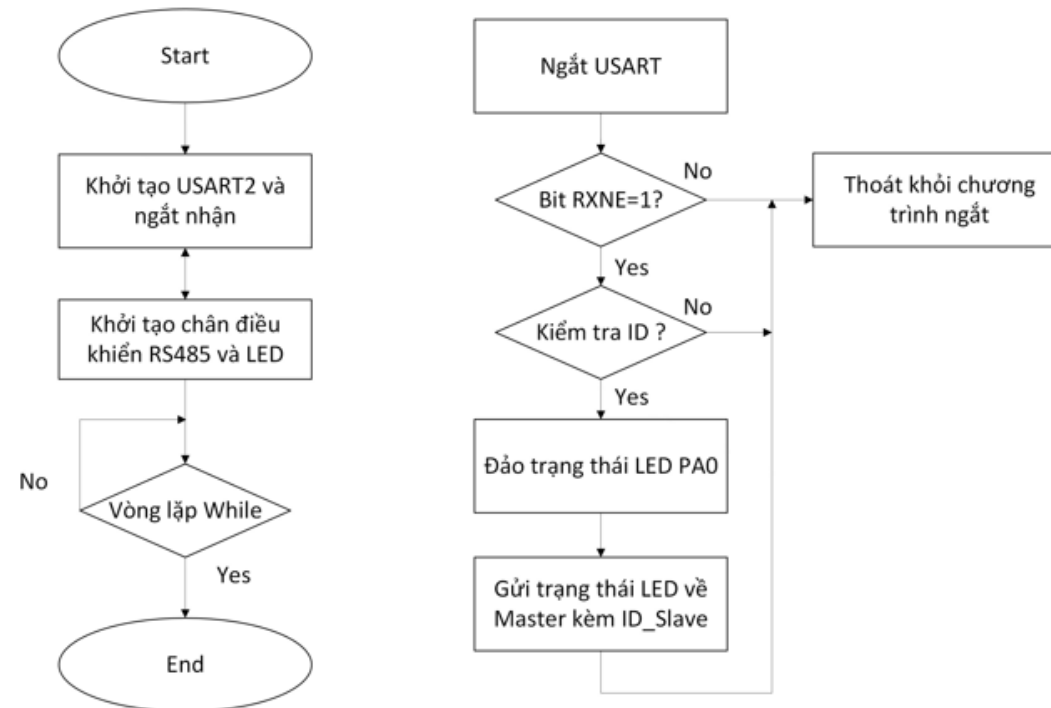
❖ Lưu đồ thuật toán của Master



Giao tiếp giữa 1 Master và 2 Slave sử dụng RS485

❖ Lưu đồ thuật toán của Slave 1

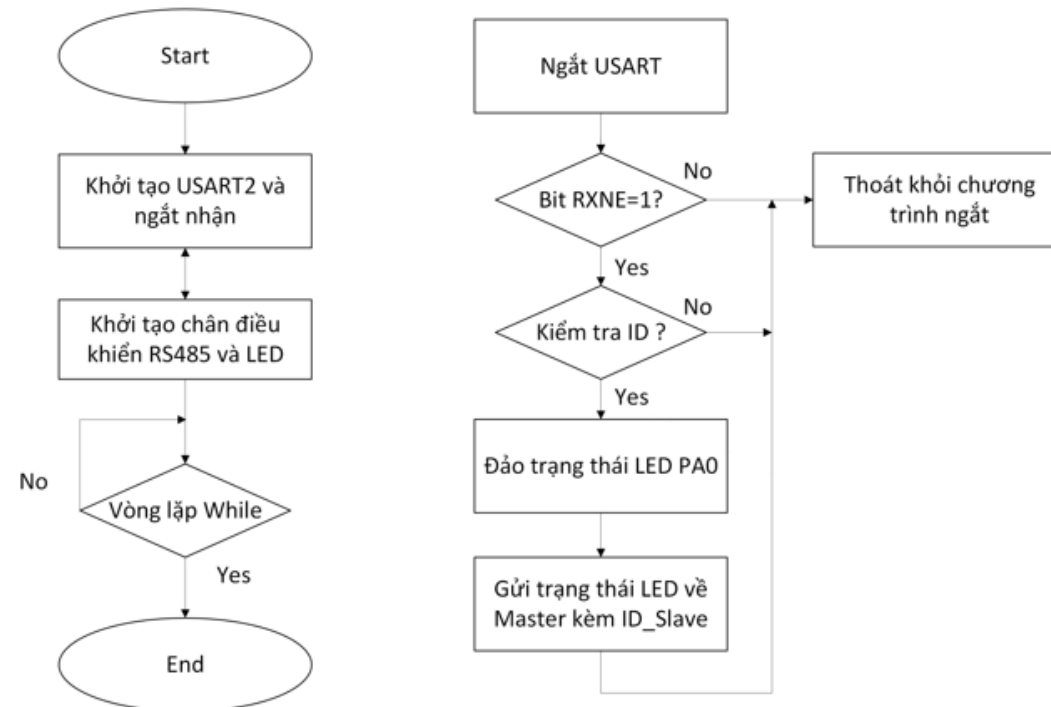
Lưu đồ thuật toán Slave truyền
nhận 3 STM32 trở lên



Giao tiếp giữa 1 Master và 2 Slave sử dụng RS485

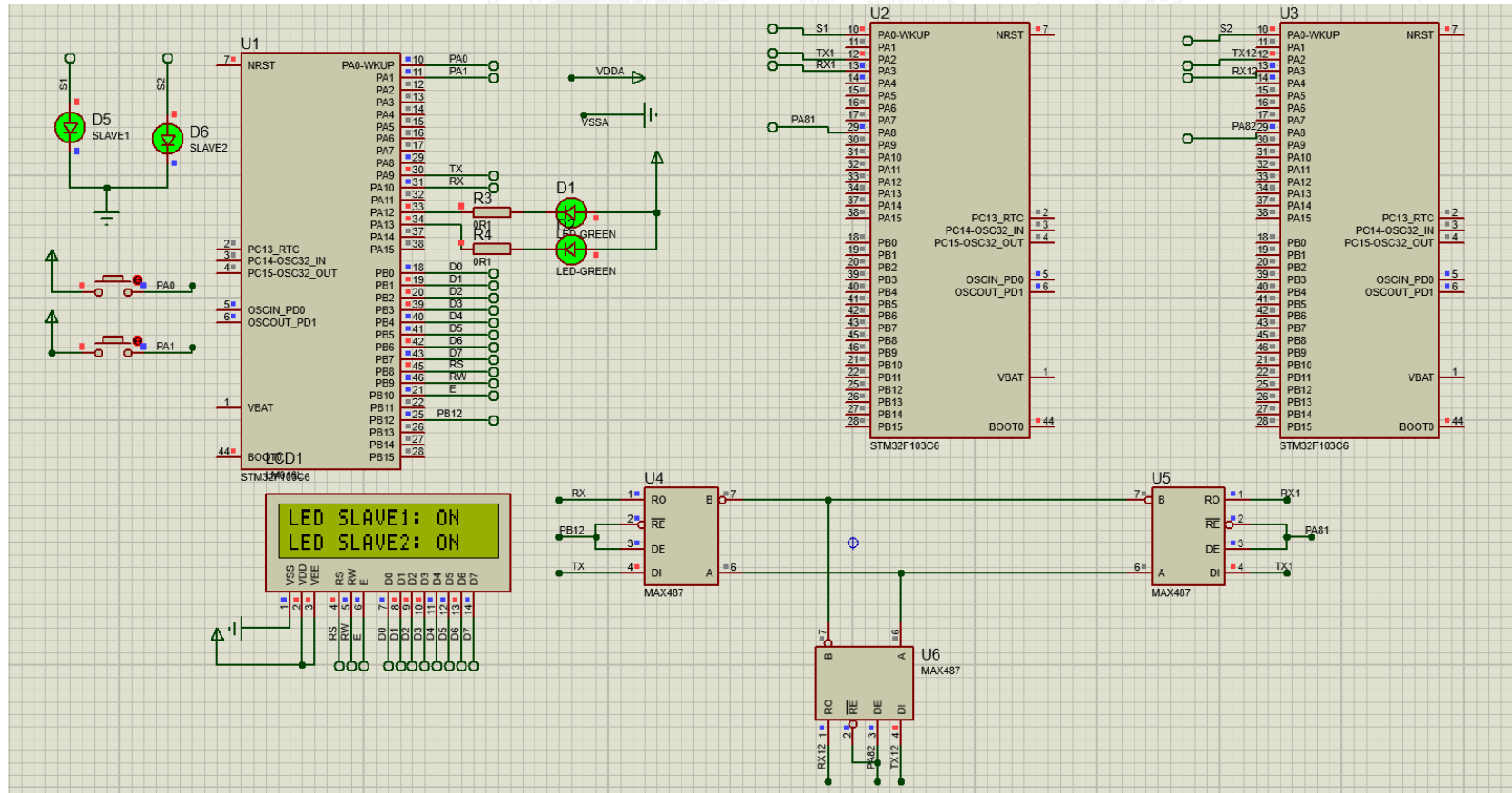
❖ Lưu đồ thuật toán của Slave 2

Lưu đồ thuật toán Slave truyền
nhận 3 STM32 trở lên



Giao tiếp giữa 1 Master và 2 Slave sử dụng RS485

❖ Kết quả



Thank You



For watching

